

Projectile motion simulation with drag force

```

1  # Projectile motion with drag force by Woramate (6/12/2568)
2  import numpy as np
3  import matplotlib
4  matplotlib.use("TkAgg")
5  import matplotlib.pyplot as plt
6  from matplotlib.animation import FuncAnimation
7
8  # Constant and values input
9  g = 9.8 # m/s^2
10 v0 = float(input("initial velocity (m/s): "))
11 theta = float(input("initial angle (degrees): "))
12 mass = float(input("mass (kg): "))
13 beta = float(input("beta ((m/s)/N): ")) # F = -(beta)v (ideal air resistance)
14 if mass == 0:
15     mass = 1 # to escape dividing by 0
16
17 theta1 = np.radians(theta) # degrees to radian
18
19 r = np.array([0.0, 0.0]) # store distance and velocity array
20 v = np.array([float(v0 * np.cos(theta1)), float(v0 * np.sin(theta1))]) # x-y compon
ent
21
22 t = 0
23 dt = 0.0422
24 steps = int(2 * v0 * np.sin(theta1) / (g * dt)) # having enough number of steps
25
26 xs = []
27 ys = []
28 ts = []
29 speeds = []
30
31 for i in range(steps):
32     xs.append(r[0])
33     ys.append(r[1])
34     ts.append(t)
35     speeds.append(np.linalg.norm(v)) # magnitude of v
36
37     if r[1] < 0:
38         break # projectile hits the floor = end
39
40     F_drag = -beta * v
41     F_grav = np.array([0, -mass * g]) # downward
42     F = F_drag + F_grav
43
44     a = F / mass
45     v = v + a * dt
46     r = r + v * dt
47
48     t = t + dt
49
50 fig, ax = plt.subplots() # animation
51 ax.set_title("Projectile Motion with Air Resistance")

```

```

52 ax.set_xlabel("x (m)")
53 ax.set_ylabel("y (m)")
54 ax.grid(True)
55
56 # screen conditions
57 if len(xs) > 0:
58     ax.set_xlim(-max(xs)/6, max(xs) * 1.2) # set initial outer limit of screen
59     ax.set_ylim(-max(ys)/6, max(ys) * 1.2)
60 else:
61     ax.set_xlim(0, 10) # in case it doesnt work, the tkAgg still runs
62     ax.set_ylim(0, 10)
63
64 if max(ys) > max(xs): # screen fitting
65     ax.set_ylim(-max(ys)/6, max(ys) * 1.2)
66     ax.set_xlim(-max(ys)/6, max(ys) * 1.2)
67 else:
68     ax.set_xlim(-max(xs)/6, max(xs) * 1.2)
69     ax.set_ylim(-max(xs)/6, max(xs) * 1.2)
70
71 # result
72 max_height = max(ys)
73 max_distance = max(xs)
74 flight_time = ts[-1] # last data
75 velo_min = min(speeds)
76
77 print(f"Maximum vertical distance: {max_height:.2f} m")
78 print(f"Maximum horizontal distance: {max_distance:.2f} m")
79 print(f"Flight time: {flight_time:.2f} s")
80 print(f"Minimum velocity: {velo_min:.2f} m/s")
81
82 max_index = ys.index(max(ys)) # points marker for ymax
83 x_ymax = xs[max_index]
84 minv_index = speeds.index(min(speeds)) # point marker for v_min
85 x_vmin = xs[minv_index]
86 y_vmin = ys[minv_index]
87
88 ax.plot(x_ymax, max(ys), marker="x", markersize = 6, color = "red")
89 ax.plot(max(xs), 0, marker="x", markersize = 6, color = "red")
90 ax.plot(x_vmin, y_vmin, marker="x", markersize = 6, color = "orange")
91
92 traject, = ax.plot([], [], lw = 1, color = "lime", label = "Trajectory")
93 point, = ax.plot([], [], 'o', markersize = 10, color = "green", label = "Projectile")
94
95 time_text = ax.text(0.03, 0.94, "", transform = ax.transAxes, fontsize = 12) # t
96 v_text = ax.text(0.03, 0.03, "", transform = ax.transAxes, fontsize = 12) # v(t)
97
98 # update frame
99 def update(frame):
100     traject.set_data(xs[:frame], ys[:frame])
101     point.set_data([xs[frame]], [ys[frame]])
102     time_text.set_text(f"t = {ts[frame]:.1f} s")
103     v_text.set_text(f"v(t) = {speeds[frame]:.2f} m/s")
104     return traject, point, time_text, v_text
105
106 ani = FuncAnimation(fig, update, frames=len(xs), interval=20, blit=True)

```

```

106
107 plt.legend()
108 plt.show()

```

Running the code

Case 1: No drag force

Value inputs:

```

initial velocity (m/s): 45
initial angle (degrees): 60
mass (kg): 0
beta ((m/s)/N): 0

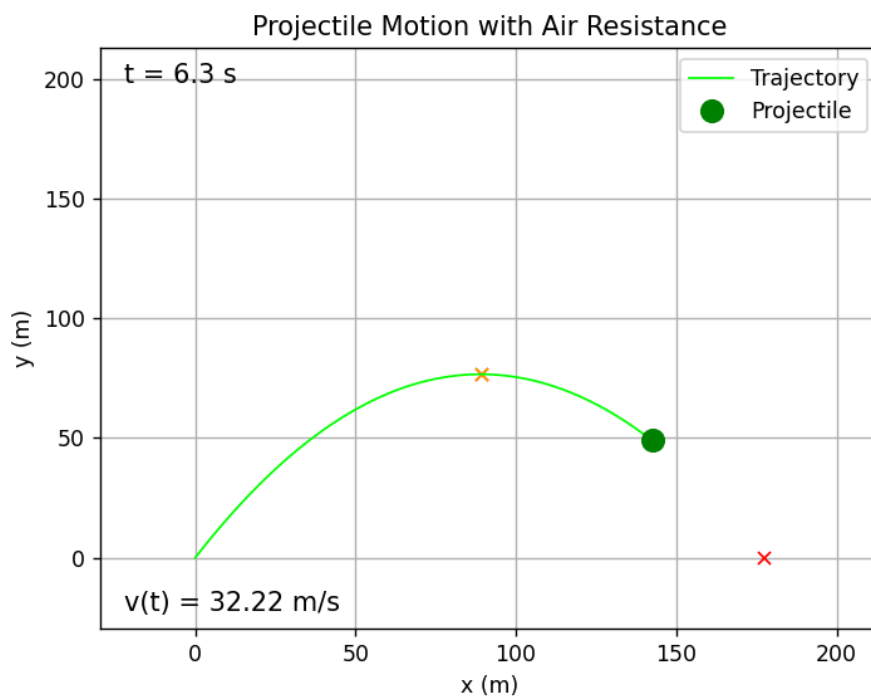
```

Outputs:

```

Maximum vertical distance: 76.67 m
Maximum horizontal distance: 177.56 m
Flight time: 7.89 s
Minimum velocity: 22.50 m/s

```



(Figure above show projectile animation in real-time and resets when the ball hit the ground.)

Lime color line forms parabolic line.

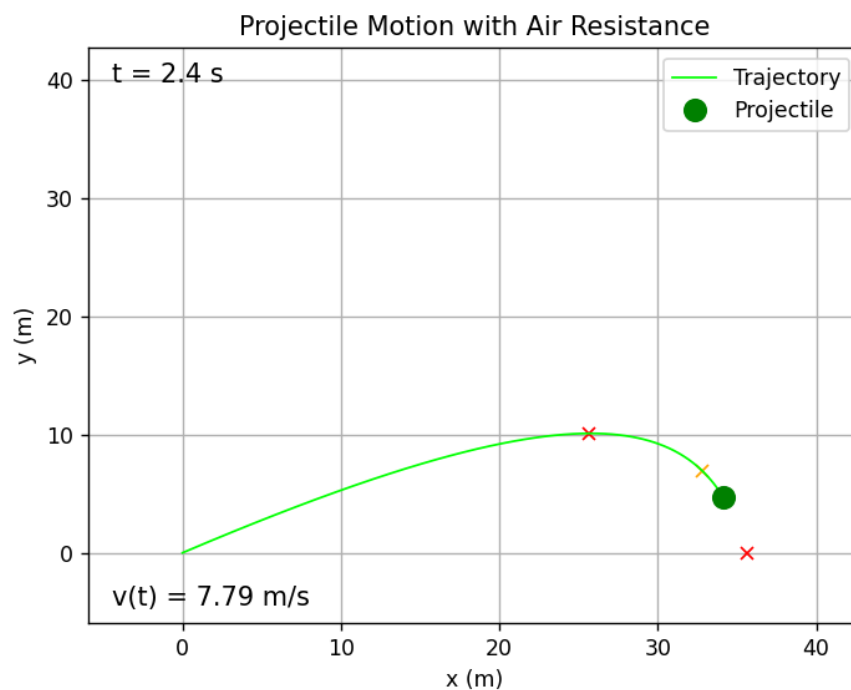
Case 2: With drag force

Value inputs:

initial velocity (m/s): 45
 initial angle (degrees): 30
 mass (kg): 5
 beta ((m/s)/N): 5

Outputs:

Maximum vertical distance: 10.11 m
 Maximum horizontal distance: 35.65 m
 Flight time: 3.04 s
 Minimum velocity: 7.55 m/s



Yellow dot shows the point with minimum velocity

Red dots show the peak and the projectile range

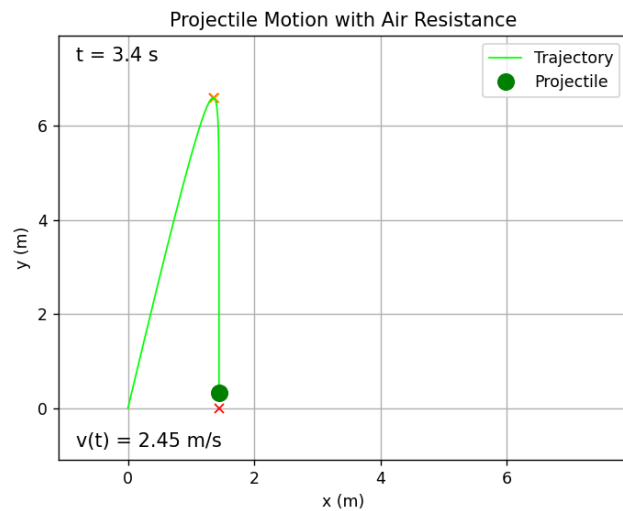
Playing with different values

Value inputs:

initial velocity (m/s): 40
 initial angle (degrees): 80
 mass (kg): 5
 beta ((m/s)/N): 20

Outputs:

Maximum vertical distance: 6.60 m
 Maximum horizontal distance: 1.44 m
 Flight time: 3.59 s
 Minimum velocity: 0.46 m/s

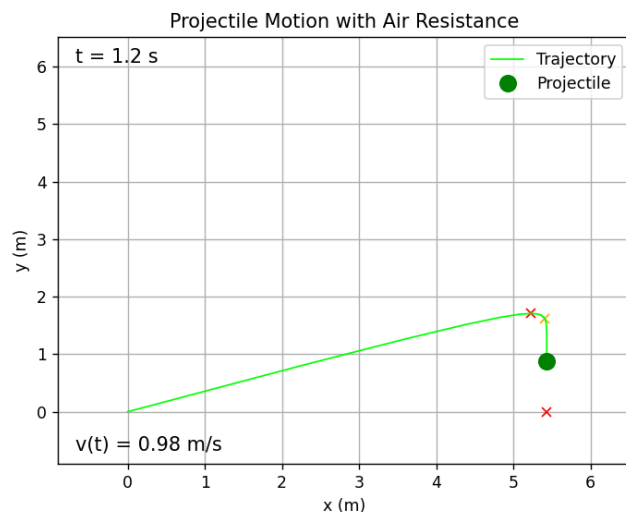


Value inputs:

initial velocity (m/s): 100
 initial angle (degrees): 20
 mass (kg): 5
 beta ((m/s)/N): 50

Outputs:

Maximum vertical distance: 1.71 m
 Maximum horizontal distance: 5.43 m
 Flight time: 2.11 s
 Minimum velocity: 0.92 m/s



Usefulness: Rockets modeling
 Badminton modeling
 Football (without spin that creates Magnus force)
 Bullets
 Analyze paths of debris falling components
 Estimate landing zones for objects dropped from height
 And much more!